Introduction to Amazon API Gateway: Hands-on Lab

Table of Contents

Lab overview

In this lab, I created a simple FAQ micro-service. The micro-service returns a JSON object containing a random question and answer pair using an **Amazon API Gateway** endpoint that invokes an **AWS Lambda** function.

The user initiates a request, which can be an HTTP or GET request, targeting services hosted within the AWS cloud. This request first reaches Amazon API Gateway, which is responsible for transforming the HTTP request into a JSON format. Subsequently, this formatted JSON is sent through a VPC endpoint to an AWS Lambda function.

The Lambda function processes the incoming request and generates a response, also in JSON format. This response is then sent back through the VPC endpoint to Amazon API Gateway, which converts it into a final HTTP response that is returned to the user.

Objectives

By the end of this lab, I was able to accomplish the following:

- create an **AWS Lambda function**,
- set up an **Amazon API Gateway** endpoint,
- and debug both **API Gateway** and **Lambda** using Amazon CloudWatch.

Lab environment

Amazon API Gateway and AWS Lambda

A microservice that uses Amazon API Gateway includes a defined resource and associated methods (such as GET, POST, PUT, etc.) within API Gateway, as well as a backend target (AWS, n.d).

In this lab, the backend target is an AWS Lambda function. However, the backend target can also be any other HTTP endpoint (like a third-party API or a web server), an AWS service proxy, or a mock integration used as a placeholder (AWS, n.d).

Services Used in This Lab
Amazon API Gateway

Amazon API Gateway is a managed service offered by AWS that simplifies the creation, deployment, and maintenance of APIs. It includes various features to enhance API functionality:

- **Request and Response Transformation:** Modify the body and headers of incoming API requests to align with backend systems, and similarly adjust the body and headers of outgoing API responses to meet API requirements.

- **Access Control**: Manage API access using AWS Identity and Access Management.

- **API Key Management**: Create and implement API keys for third-party developers.

- **Monitoring**: Integrate with Amazon CloudWatch to monitor API performance.

- **Caching**: Utilize Amazon CloudFront to cache API responses for improved response times.

- **Multi-Stage Deployment**: Deploy an API to multiple stages, facilitating differentiation between development, testing, production, and versioning.

- **Custom Domain Connection**: Connect custom domains to your API.

- **Model Definition**: Define models to standardize transformations of API requests and responses.

AWS Lambda

Is a powerful serverless, event-driven compute service that enables you to execute code without the need for server management. It works seamlessly with over 200 AWS services and third-party applications, allowing you to trigger functions through various events like user actions on an e-commerce site. This flexibility means you only pay for the compute time you actually use.

With AWS Lambda, you can enhance existing AWS services with custom logic, create scalable backend solutions, and automate administrative tasks. The service supports a range of features including built-in fault tolerance, automatic scaling, and the ability to package functions as container images.
You can also connect Lambda functions to relational databases and shared file systems, orchestrate multiple functions, and integrate it with AWS Identity and Access Management (IAM) for a secure environment. Plus, monitoring and observability tools help you keep track of your applications effectively.

Task 1: Create a Lambda Function

In this task, I used the AWS Lambda. Console to create a function with lab provided code, and configured it to work with Amazon API Gateway.

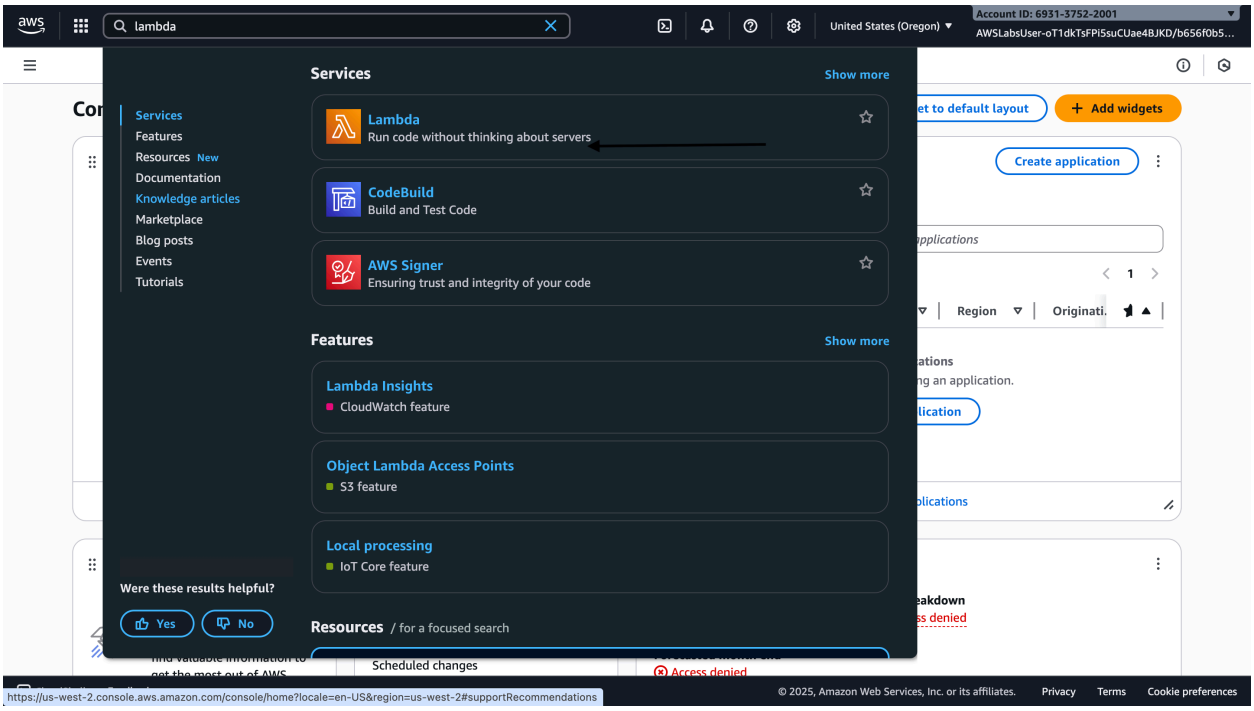

Figure 1: Lambda console (source: personal collection)

Task 1.1: Create the initial Lambda Function

- Step 1: Select “Author from scratch”
- Step 2: set “Function name” to FAQ
- Step 3: set “Runtime” to “Node.js 18.x”
- Step 4: “Change default execution role” |
- Step 5: “Use an existing role” inside “Execution role”
- Step 6: “choose VPC”
- Step 7: select VPC with CIDR range 10.0.0.0/16
- Step 8: select two subnets with CIDR: 10.0.1.0/24, 10.0.2.0/24
- Step 9: “Create function”

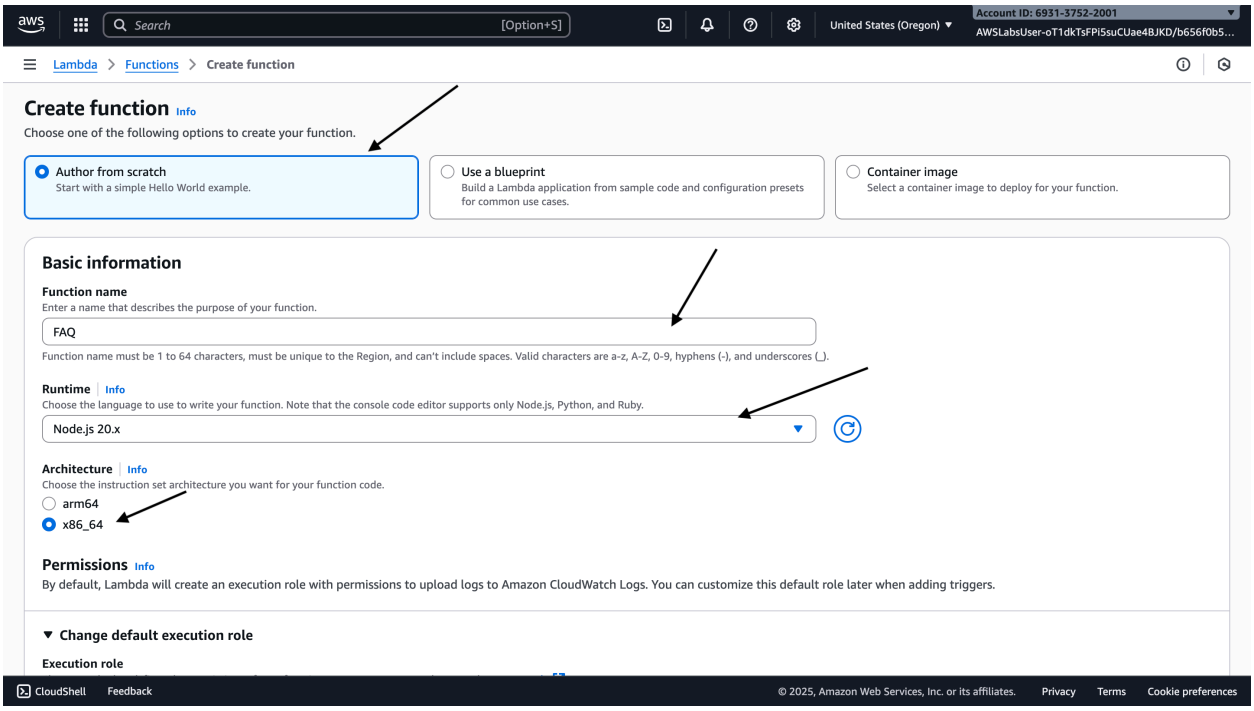

Figure 2: Create function page (source: personal collection)

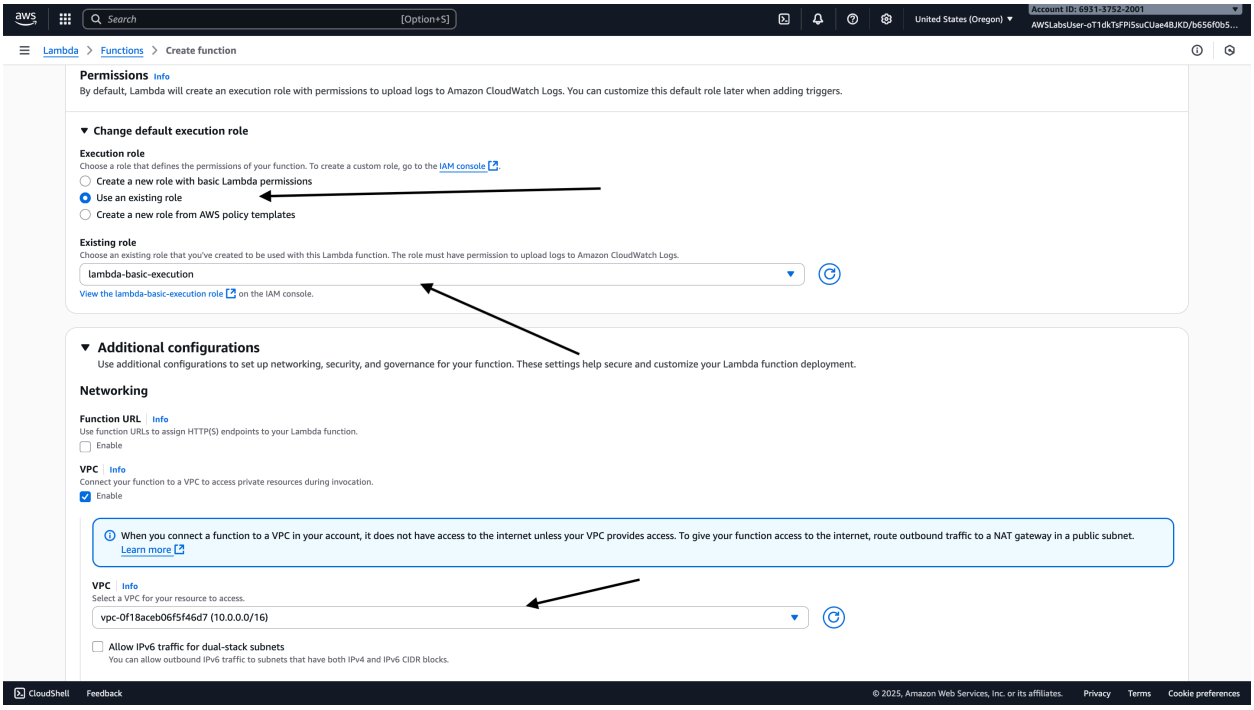

Figure 3: Additional Configuration tabs (source: personal collection)

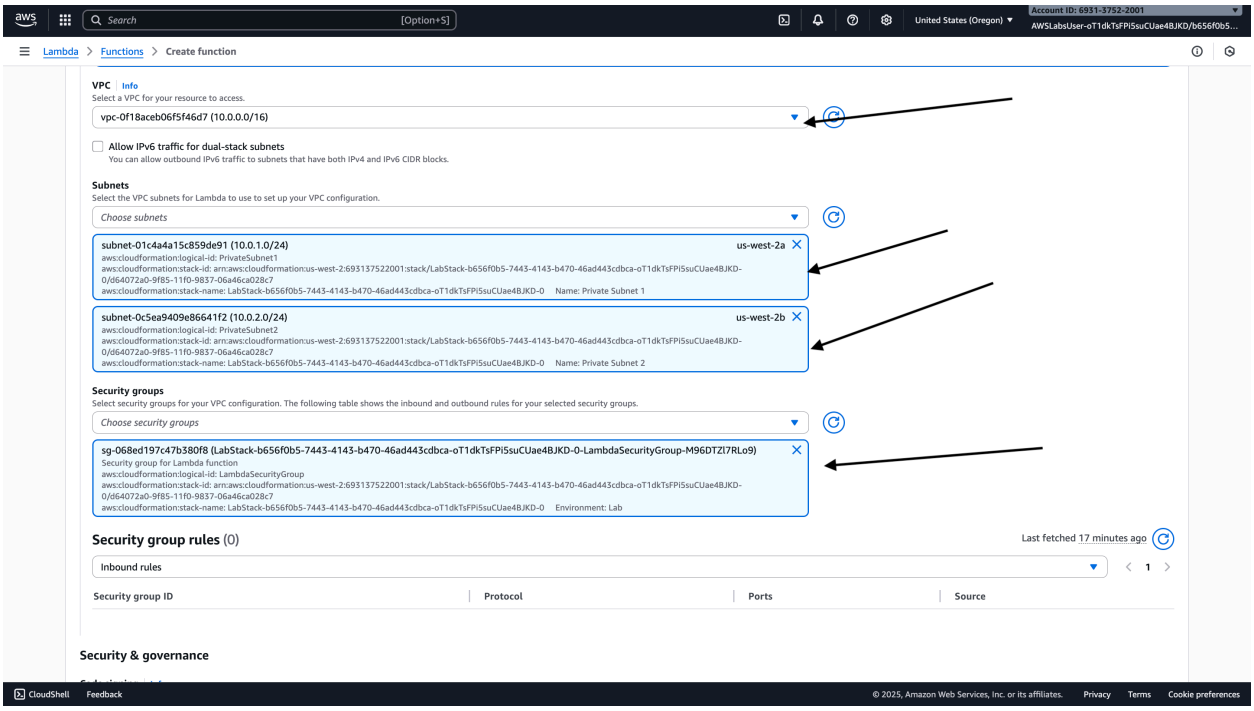

Figure 4: Subnets & Security groups configuration (source: personal collection)

- Step 10: set up the JSON codes
- Step 11: deploy the code

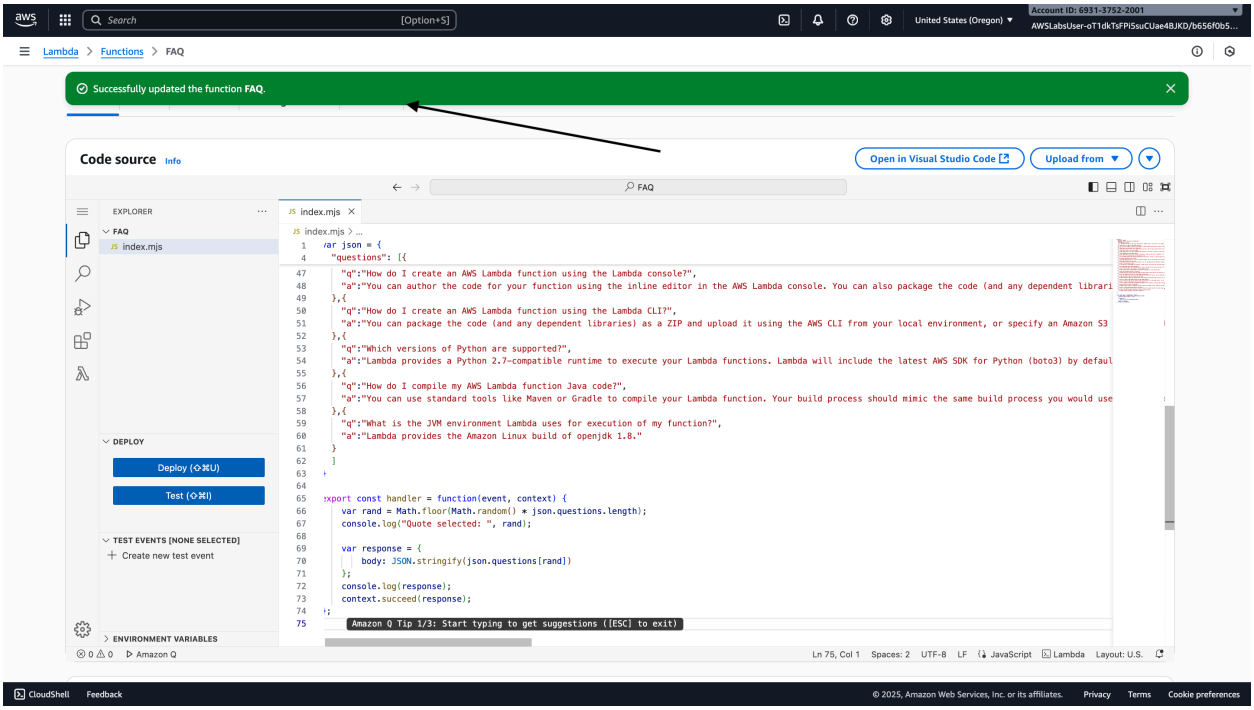

Figure 5: Deployment success (source: personal collection)

Task 2: Create an API Gateway endpoint

An API endpoint is a hostname of the API. This can be edge-optimized or regional, depending on where the traffic originates from.

- Step 1: choose the “Configuration” tab
- Step 2: choose “General configuration”, then “edit”
- Step 3: select “Description”
- Step 4: “save”

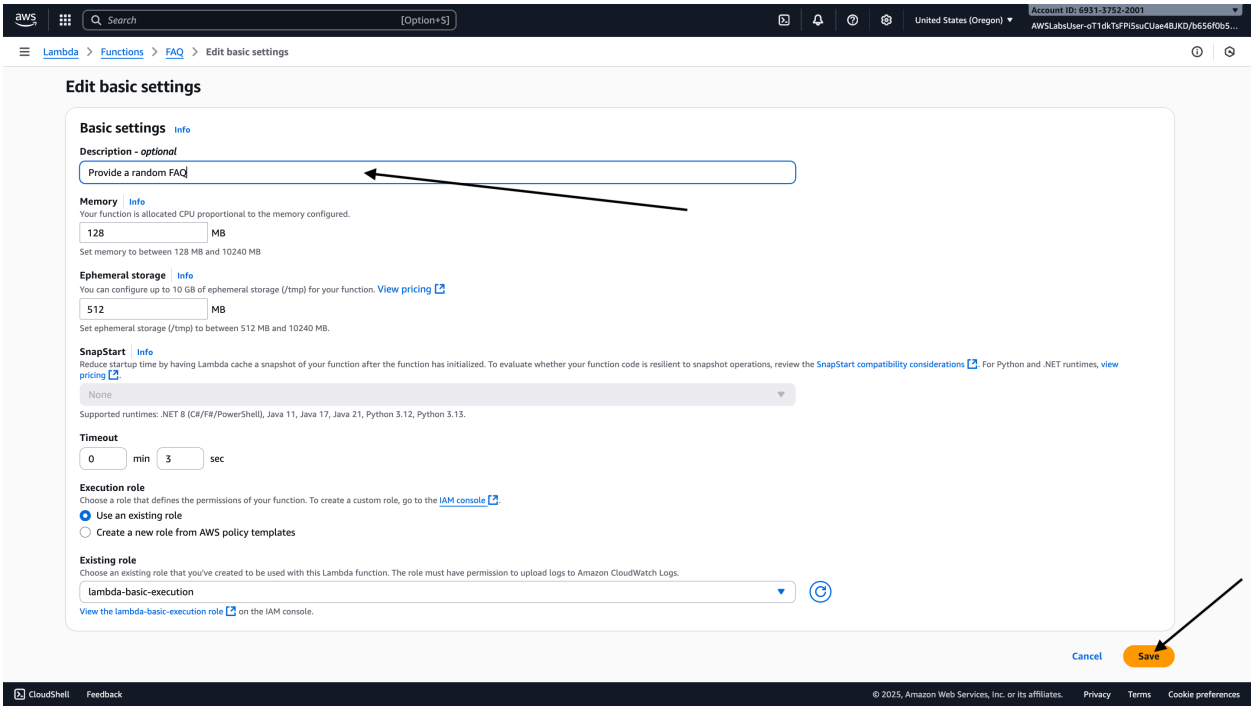

Figure 6: General Configuration tab (source: personal collection)

- Step 5: select “API Gateway” as the source
- Step 6: select “Create a new API”
- Step 7: select “REST API” as “API type”
- Step 8: select “API name”
- Step 9: set “Deployment stage”
- Step 10: choose “add”

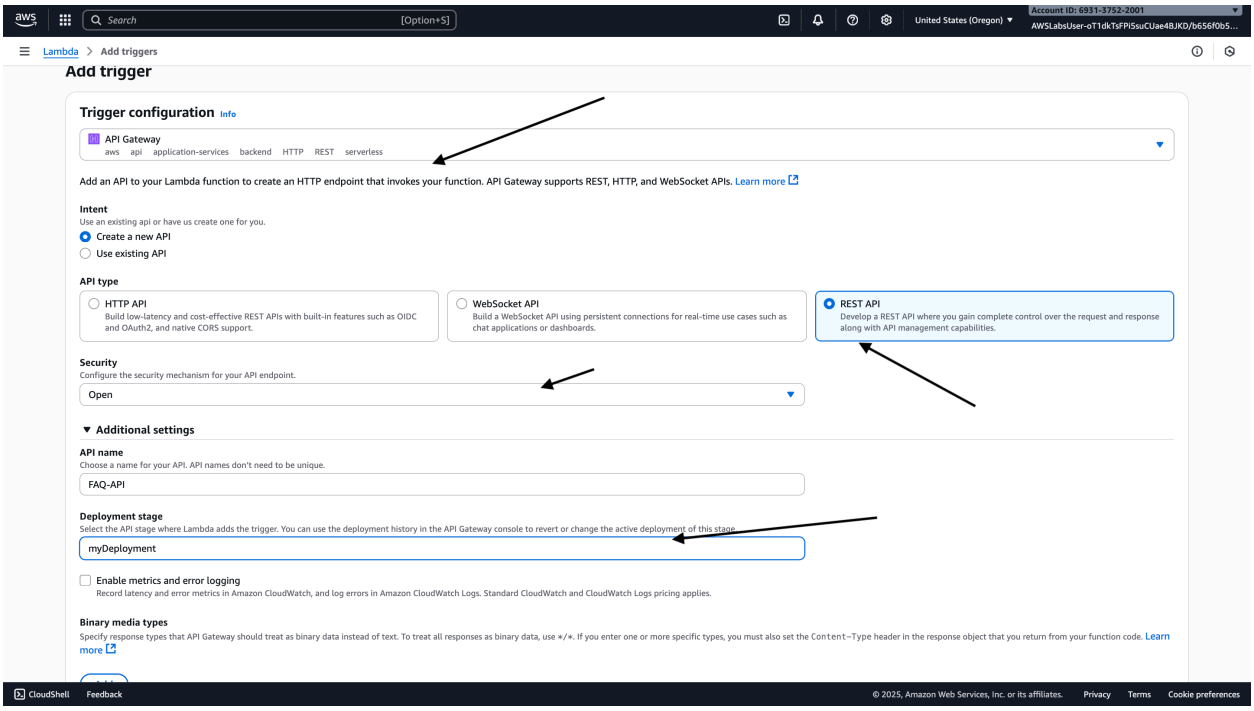

Figure 7: Add trigger page (source: personal collection)

Task 2: Test the Lambda function

- Step 1: select “Triggers” from the navigation menu on the left
- Step 2: select “API Gateway” and expand the “details”
- Step 3: open the “API endpoint” on the browser

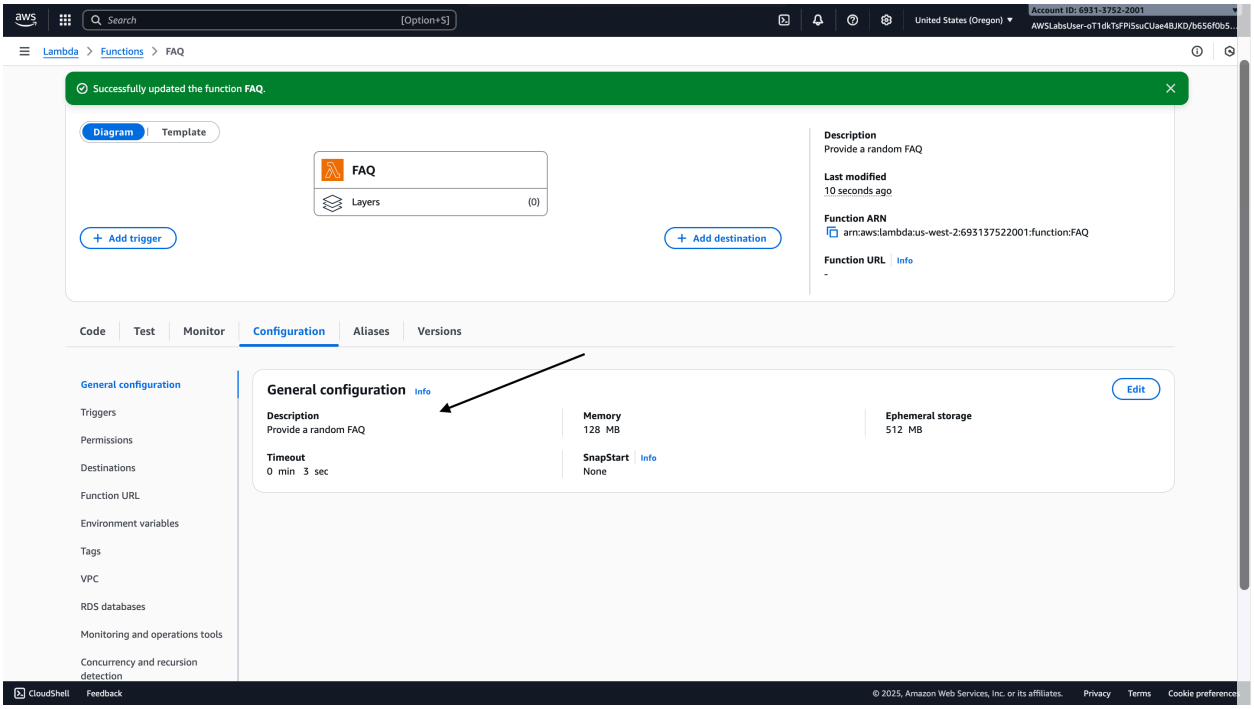

Figure 8: General configuration (source: personal collection)

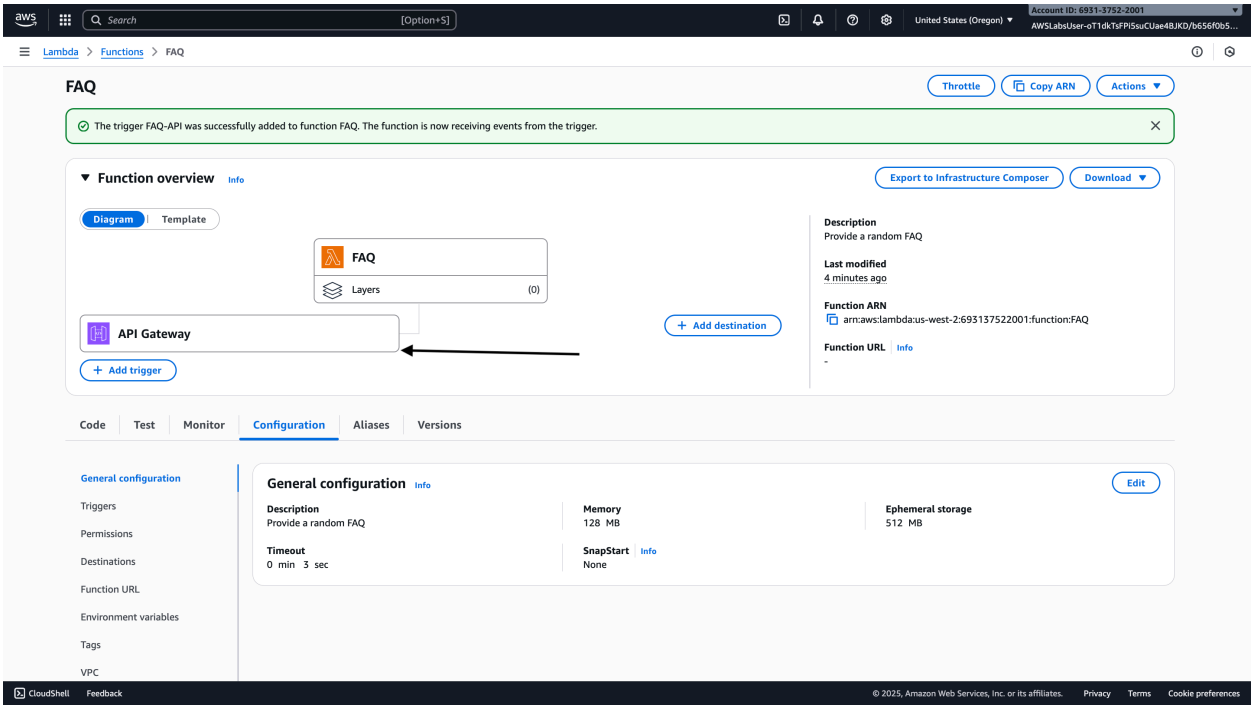

Figure 9: API Gateway (source: personal collection)

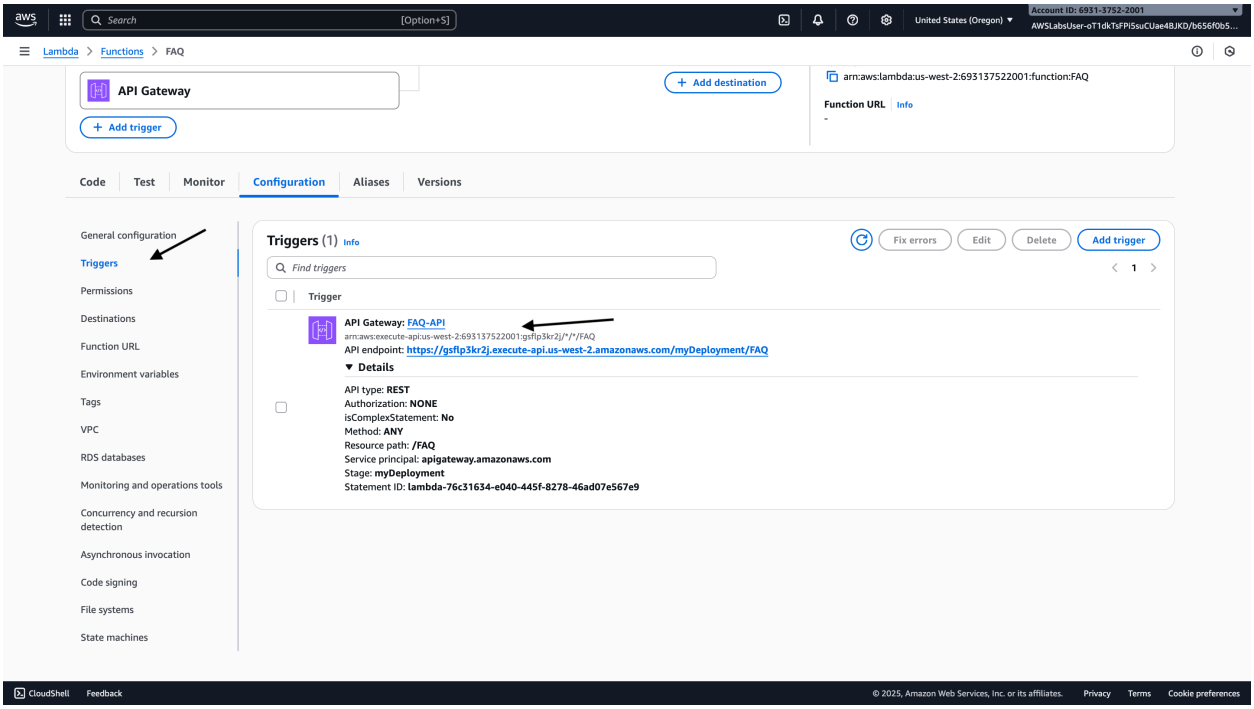

Figure 10: Triggers (source: personal collection)

- Step 4: Configure a new test event “BasicTest”
- Step 5: keep the JSON object empty
- Step 6: Save
- Step 7: Test
- Step 8: verify the execution result’s details
- Step 9: Access the “Monitor” tab
- Step 10: choose “Cloudwatch logs” and analyze the log streams

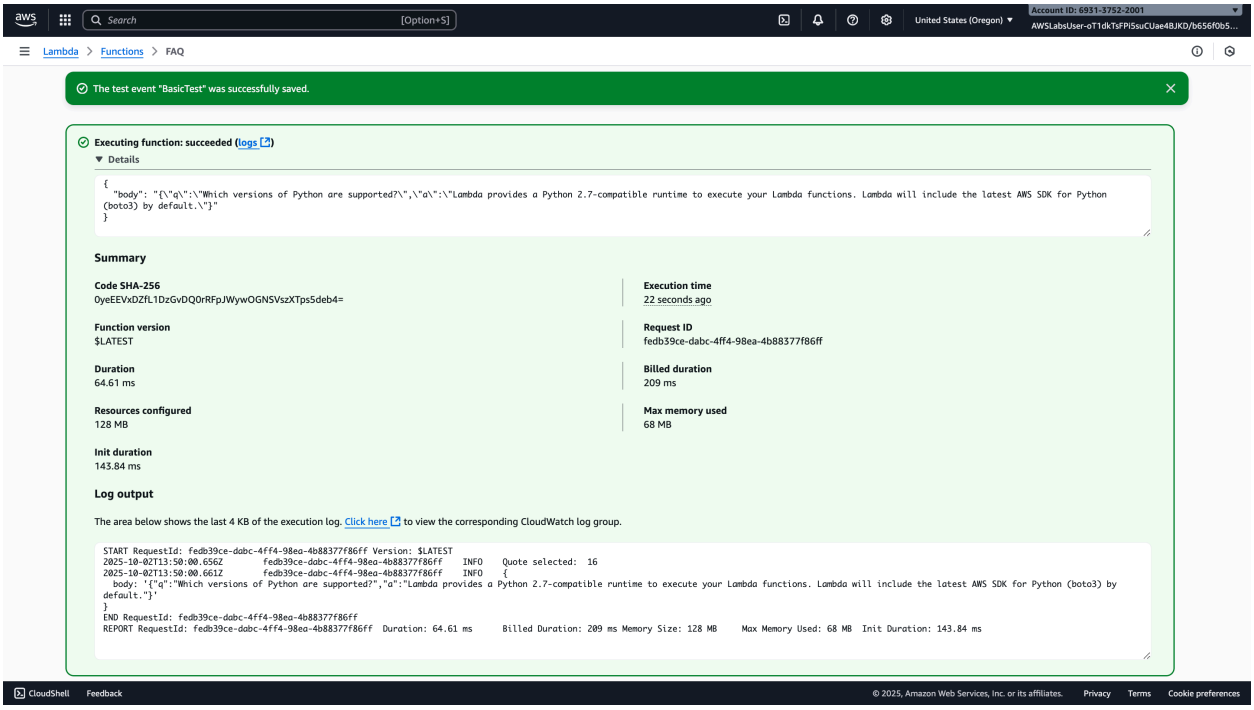

Figure 11: Test results (source: personal collection)

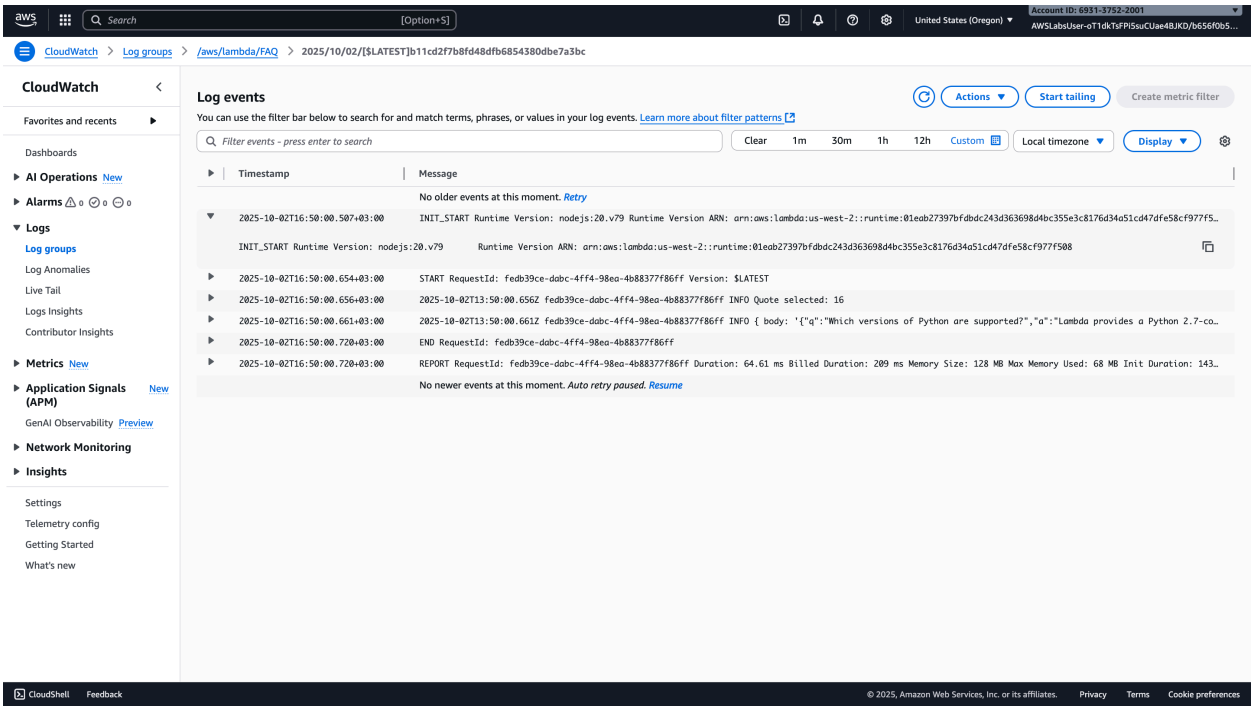

Figure 12: Cloudwatch page (source: personal collection)

Conclusion

In this lab, I explored how to create a microservice using a Lambda function, then connect it to the API Gateway endpoints, and finally Debug API Gateway and Lambda with Amazon CloudWatch.

Reference

Amazon Web Services. (n.d.). *Introduction to Amazon API Gateway*. Retrieved from https://lab.builder-labs.skillbuilder.aws/sa/lab/arn%3Aaws%3Alearningcontent%3Aus-east-1%3A470679935125%3Ablueprintversion%2Fspl-58%3A2.0.34-68467076/en-US/b656f0b5-7443-4143-b470-46ad443cdbca::oT1dkTsFPi5suCUae4BJKD